

A Fog Computing Pipeline for Data Center Environmental Monitoring: Design, Deployment, and Evaluation on AWS

Nithin

Student ID: X25125338

MSc in Cloud Computing

Fog and Edge Computing (H9FECC)

National College of Ireland

x25125338@student.ncirl.ie

Abstract—This paper presents the design, implementation, and real-world cloud deployment of a fog and edge computing pipeline for data center environmental monitoring. The system simulates five distinct sensor types (temperature, humidity, airflow, power load, and dust density) across two server halls, each independently configurable for sampling and dispatch frequency. A Node.js fog node buffers incoming readings in a fixed-size ring buffer, performs windowed aggregation and threshold-based alerting, and dispatches each window's results as a single batched message. The backend is entirely serverless and split across two independently deployed AWS Lambda functions: the first is triggered directly by an Amazon SQS queue and writes aggregated windows to Amazon DynamoDB, while the second performs its own internal request routing and sits behind a real Amazon API Gateway REST API, rather than a dashboard server calling AWS services directly. The dashboard's static assets are served from Amazon S3 and call the API Gateway endpoint directly. The complete system was deployed and tested on a real AWS account rather than a local emulator alone, which surfaced defects invisible to local testing, including credential-handling bugs that only manifest under genuine temporary AWS credentials and a DynamoDB pagination bug that silently undercounted large table scans. This paper critically analyses the architectural decisions taken at each layer, the defects discovered during real deployment, and the lessons learned for building genuinely cloud-native fog computing systems.

Keywords—fog computing, edge computing, Internet of Things, serverless architecture, AWS Lambda, cloud deployment, data center monitoring

I. INTRODUCTION

Data centers depend on continuous environmental monitoring to protect the hardware they house: an undetected overheating server, a humidity swing that risks condensation on sensitive equipment, restricted airflow around a rack, an unexpected power-load spike, or rising dust density between scheduled inspections can each trigger a hardware failure or a costly unplanned outage, precisely the class of condition wireless sensor deployments in real data centers have been built to catch before it escalates [1], [2]. A facility relying on a fixed inspection schedule rather than continuous sensing discovers these conditions only after the fact, precisely when a condition-driven system is meant to surface them immediately. Internet of Things sensing, combined with fog and edge processing close to the sensors themselves, allows a facilities team to evaluate every reading against explicit threshold rules as it arrives, rather than waiting for the next scheduled walk-through.

The National Institute of Standards and Technology defines cloud computing around five essential characteristics:

on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service [3]; fog computing extends that model rather than replacing it. Bonomi et al. define fog computing as a platform that extends cloud services to the edge of the network, characterised by low latency, geographic distribution, and support for a large number of nodes [4]. This project adopts that definition directly: windowed aggregation and threshold evaluation are performed on a fog node co-located with the sensors, and only the resulting aggregate-plus-alerts payload, not every raw reading, crosses the network boundary into the cloud.

This paper documents the design and construction of a fog and edge computing pipeline for this domain. The system was designed to satisfy three requirements at minimum. First, a sensor and fog layer generating data from five distinct sensor types with independently configurable sampling and dispatch rates, where the fog node genuinely buffers, aggregates, and evaluates sensor data before dispatching it onward. Second, a scalable backend layer, built from managed queue and function-as-a-service mechanisms, that processes the fog node's output and exposes a responsive dashboard for the sensor types. Third, deployment and testing of that backend on a public cloud platform, not merely a local simulation.

The remainder of this paper is organised as follows. Section II describes the system's architecture and the design decisions taken at each layer, including the separately deployed API backend that distinguishes this project's architecture, a critical comparison against the alternatives that were not chosen, and the security posture of the resulting deployment. Section III describes the concrete implementation: the software components and libraries used, the automated test suite, the continuous integration pipeline, and the real AWS deployment together with the genuine defects it surfaced. Section IV evaluates the deployed system against measured evidence rather than intent alone. Section V concludes with a critical reflection on the project and directions for future work.

II. SYSTEM ARCHITECTURE AND DESIGN

A. Sensor and Fog Layer

The simulated deployment models two server halls, hall-1 and hall-2, each instrumented with five sensor types: temperature_c, humidity_pct, airflow_cfm, power_load_kw, and dust_density_ugm3. Every one of the ten resulting sensor processes exposes two independent, separately configurable environment variables, SAMPLE_INTERVAL and DISPATCH_INTERVAL, so that generation frequency and dispatch rate are genuinely decoupled knobs rather than a single aliased value.

The fog node buffers incoming readings in a fixed-size ring buffer keyed by (sensor_type, site_id): each key holds a plain array of RING_CAPACITY (256) slots with a manually tracked write index that advances modulo the capacity, a bounded circular-buffer structure for streaming data rather than an unbounded growing list [5], [6]. On every window tick, the fog node reconstructs each ring's contents in oldest-first order, computes the window's minimum, maximum, average, latest value, and reading count, evaluates the aggregate against the domain-specific threshold rules listed in Table I, and resets the ring for the next window. The resulting aggregate-plus-alerts payload is dispatched to an Amazon SQS queue through an EventEmitter-decoupled publisher: window-close and the SQS send are two separate concerns joined only by an emitted event, echoing the general publish-subscribe pattern of separating event producers from consumers in space, time, and synchronisation [7], [8]. The publisher always issues a batched SendMessageBatchCommand, chunked at SQS's ten-entry ceiling, so every group closed in one flush cycle goes out together whenever more than one is ready.

TABLE I. ENVIRONMENTAL ALERT RULES

Sensor Type	Rule	Alert Key
temperature_c	avg > 27°C	overheat_risk
humidity_pct	avg > 60%	condensation_risk
humidity_pct	avg < 20%	static_discharge_risk
airflow_cfm	avg < 400 CFM	insufficient_cooling
power_load_kw	avg > 130 kW	capacity_warning
dust_density_ugm3	avg > 50 ug/m3	air_quality_risk

B. Scalable Backend Layer

The backend layer is built entirely from managed, scalable AWS primitives. An Amazon SQS standard queue (dce-hall-agg) decouples the fog node's dispatch rate from the backend's processing rate, and an AWS Lambda function (dce-processor), subscribed to that queue via an event-source mapping, is invoked automatically as messages arrive, writing each aggregated window into an Amazon DynamoDB table (dce-readings). The table uses sensor_type as its partition key and a composite sort key of the form window_end#site_id as its range key, following the same partition-key philosophy behind Amazon's own eventually-consistent, highly available key-value store: a narrow, well-known set of key-based lookups rather than ad-hoc relational queries [9]. This design choice is deliberate: if the sort key were window_end alone, hall-1 and hall-2 producing an aggregate for the same sensor type in the same flush cycle would collide on an identical primary key, and whichever write landed second would silently overwrite the first. Appending the site identifier to the sort key disambiguates the two halls' concurrent writes without requiring any additional index.

A commonly raised critique of Lambda-based architectures is cold-start latency, an initialisation delay incurred whenever an invocation finds no currently warm execution environment, a cost documented empirically across commercial FaaS platforms [10]. This is a genuine cost of the FaaS approach, but is acceptable here for the same reason it is acceptable for the ingestion path: the fog-to-processor path is asynchronous and queue-triggered, not a user-facing request/response path, so an occasional cold-start delay when a reading is durably stored is not a user-visible page load.

C. A Separately Deployed API Backend Behind a Real API Gateway

Unlike a design in which the dashboard's own backend process calls DynamoDB, SQS, and Lambda directly, this project's individually required architecture deploys a second, entirely separate Lambda function, dce-api, that performs its own internal path-and-method routing against an ordered [method, regex, handler] table, directly unit-testable with fake AWS clients and no Lambda runtime at all. This function is fronted by a real Amazon API Gateway REST API: a single {proxy+} resource under the root, with an ANY-method AWS_PROXY integration, so every request the dashboard makes to any /api/* path is proxied verbatim to dce-api, which then decides internally how to answer it. This centralises cross-cutting request handling behind a single gateway rather than exposing each backend capability as its own directly reachable endpoint [11]. dce-api answers five endpoints: GET /api/readings, GET /api/halls (and /api/halls/:hallId for a single hall), GET /api/health, GET /api/backend-stats, and GET /api/thresholds, the last proxied from the fog node's own /thresholds endpoint so a client can retrieve the live alert rules without duplicating them.

D. Dashboard Layer and Its Real-AWS Adaptation

The dashboard was designed from the outset as a deliberately minimal reverse proxy rather than a REST API of its own: its Node.js HTTP server resolves the API Gateway's invoke URL exactly once at startup and forwards every /api/* request to it verbatim, holding no AWS SDK imports and no route table of its own. Locally, this same server also serves the dashboard's static assets on a single port. For the real AWS deployment, this reverse-proxy role was retired rather than ported to a second EC2 process: the dashboard's HTML, CSS, and JavaScript are instead served directly from an Amazon S3 bucket over the bucket's own HTTPS endpoint, and the frontend calls the API Gateway's invoke URL directly, configured through a single meta tag rather than a hard-coded value, with the dce-api Lambda's own JSON responses carrying an Access-Control-Allow-Origin header for the resulting cross-origin request.

This mirrors a constraint discovered during a separate project in this portfolio: a raw EC2 public IP address on a non-standard port served over plain HTTP is exactly the traffic shape that campus and corporate WiFi networks commonly block by policy, so hosting the dashboard on EC2 at all would have reintroduced a reachability risk that the fully serverless path avoids entirely.

E. Deployment Topology

Fig. 1 summarises the resulting topology. Sensor processes and the fog node run in Docker containers on a single EC2 instance, managed entirely through AWS Systems Manager Session Manager rather than SSH, so no key pair is provisioned and no inbound SSH port is opened on the instance's security group. The fog node dispatches aggregated windows to an SQS queue, which triggers dce-processor, which writes to DynamoDB. dce-api, sitting behind API Gateway, reads from that same table (and from the queue, dce-processor's own metadata, and the fog node's health/thresholds endpoints) to answer the dashboard's REST API calls, while the dashboard's static frontend is served independently from Amazon S3, reached directly by the browser. This topology places every backend component except the sensor and fog tier behind a managed, serverless service boundary.

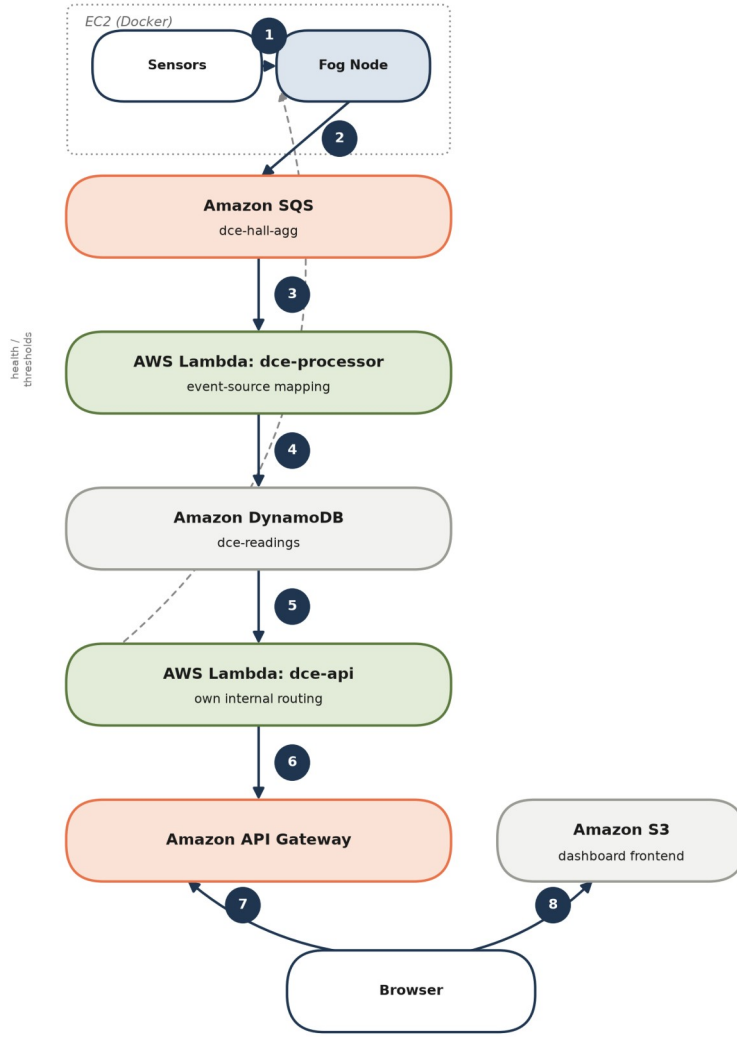


Fig. 1. Deployment topology: sensors and the fog node run on EC2; the backend (SQS, both Lambda functions, DynamoDB, API Gateway) is fully serverless; the frontend is served from S3.

F. Critical Analysis of Alternative Architectures

Several alternative designs were considered and rejected, each for a specific, statable reason. A single Lambda function handling both queue-triggered ingestion and API-Gateway-fronted request serving was considered in place of two separate functions, and rejected: ingestion and query serving have different invocation triggers and different concurrency and timeout characteristics, and coupling them in one function would mean a slow or failing query path could starve the ingestion path of concurrency, or vice versa, exactly the failure mode splitting the two functions is meant to avoid.

An unbounded, per-key growing array was considered in place of the fixed-size ring buffer for fog-side buffering, and rejected in favour of a genuinely bounded structure: an unbounded buffer risks unconstrained memory growth if a downstream flush is ever delayed, whereas a fixed-capacity ring buffer with a wrapping write index guarantees a hard upper bound on memory use regardless of how long a window runs, at the cost of silently discarding the oldest readings once that bound is exceeded, an accepted trade for a workload whose window length is short relative to its sampling rate.

A synchronous, directly-calling dashboard backend, the pattern the rest of this portfolio's Node.js siblings use, was considered in place of the decoupled, API-Gateway-fronted dce-api Lambda, and rejected specifically because this

project's individual rubric required an architecture that differs structurally, not merely superficially, from a single always-running backend process that imports the AWS SDK directly.

Finally, converting the fog node and sensors themselves to a scheduled-Lambda model, so that the entire system, not only the backend, would be serverless, was considered and explicitly deferred rather than rejected outright. A continuous sensor loop and a stateful ring buffer both map awkwardly onto Lambda's stateless, time-limited invocation model without a redesign of the buffering logic itself, and this project's cloud-deployment scope is also limited to the backend layer rather than the sensor and fog layer. Pursuing this conversion was judged a larger redesign than the remaining project time justified, and is recorded as future work in Section V.

G. Security Considerations

The EC2 instance is administered exclusively through AWS Systems Manager Session Manager; no key pair was ever provisioned, and its security group has no inbound rule for port 22, so there is no SSH attack surface at all. The only inbound rule opened is for the fog node's own health and threshold endpoints (port 8000), which dce-api needs to reach, and it exposes only read-only, non-sensitive status information rather than data or control-plane access.

Both Lambda functions execute under the AWS Academy Learner Lab account's single shared LabRole, since this account's service-control policies prevent a student from creating new, more narrowly scoped IAM roles. This is an actual limitation, not a design preference: in a production account, dce-processor and dce-api would each hold their own least-privilege role (dce-processor needing only `sqs:ReceiveMessage/DeleteMessage` and `dynamodb:PutItem`; dce-api needing only read-oriented `DynamoDB`, `SQS`, and `Lambda-metadata` actions), and this gap is recorded honestly here rather than glossed over. The dashboard's own REST API is intentionally unauthenticated, since it exposes only aggregated, non-personal environmental readings for a facility, not data for which access control would be a meaningful requirement at this project's scale.

III. IMPLEMENTATION

A. Software Components and Libraries

The sensor, fog, processor, and dce-api components are all implemented in Node.js 20 using plain CommonJS (no TypeScript build step), the official AWS SDK for JavaScript v3 for every AWS interaction, and Node's own built-in http module rather than a web framework such as Express, keeping the dependency surface deliberately small. The dashboard's trend visualisation is rendered client-side with Chart.js, vendored locally rather than loaded from a content-delivery network so the page has no third-party runtime dependency. Local development and continuous integration both use Docker Compose together with LocalStack, an open-source AWS cloud emulator, so the same `SQS`, `DynamoDB`, and `Lambda` interactions used in production can be exercised without touching a real AWS account during day-to-day development.

B. Testing Strategy

Every module has its own automated test suite written with Node's built-in test runner, exercised through hand-written fake AWS client objects rather than mocking libraries, so each unit test asserts on the exact request shape sent to a given AWS SDK call. The ring buffer's own test suite specifically covers wraparound behaviour, including a case where writes exceed capacity mid-window and only the last `RING_CAPACITY` readings survive, and that every ring's slots, write index, and count are reset to empty on every window tick. The EventEmitter-decoupled publisher is tested for exactly one `SendMessageBatchCommand` call carrying multiple `Entries` when more than one group closes in the same flush cycle. dce-api's internal router is tested by dispatching method-and-path-regex matches against fake AWS clients directly, with no `Lambda` runtime or `API Gateway` involved at all.

Beyond unit tests, a load-testing script bursts a configurable number of synthetic messages directly onto the real queue. It makes two assertions: first, that the queue depth is non-zero immediately after the burst, proving the messages genuinely reached `SQS` rather than being silently dropped; second, that the queue depth has either fully drained or measurably decreased within a timeout window. This second, timeout-tolerant check allows for the underlying `Lambda` concurrency ceiling without treating a slower-than-expected drain as a hard failure.

C. Continuous Integration and Deployment

A GitHub Actions workflow runs the full automated test suite for all five modules on every push to the repository. A second job then brings the entire stack up with Docker Compose against LocalStack and runs a dedicated end-to-end

verification script that exercises the live pipeline, submits synthetic readings, and asserts that they land in `DynamoDB` with the expected `sort_key` shape. The stack is torn down afterwards regardless of outcome, so a failed run cannot leave a stale container running for the next one.

A second, separately triggered GitHub Actions workflow exists specifically to restart the EC2-hosted fog and sensor tier once an AWS Academy Learner Lab session ends and a fresh one begins, since the Learner Lab platform stops the EC2 instance between sessions. It reads its AWS credentials from encrypted GitHub repository secrets, verifies that the credentials resolve to the expected AWS account before doing anything else, and only then starts the instance, waits for its Systems Manager agent to register, and re-verifies that the fog node is reachable both directly and through dce-api's own gateway field before reporting success.

D. Real AWS Deployment and Defects Discovered

The complete system, including the sensor and fog tier, the `SQS` queue, both `Lambda` functions, the `DynamoDB` table, the `API Gateway` endpoint, and the `S3`-hosted frontend, was deployed to a real AWS account (an AWS Academy Learner Lab account) in region `us-east-1`, so that the backend was deployed and tested on an actual public cloud platform rather than only a local emulator. This real deployment was directly responsible for surfacing two classes of defect that the local, `LocalStack`-backed test suite had not caught.

First, three separate files across the fog, processor, and dce-api components constructed their AWS SDK client credentials using a check that was true in both the local and the real-AWS case: whenever an AWS access key environment variable was present, the code applied a hardcoded, `LocalStack`-only static credential pair, in one file unconditionally, with no check at all. Because real AWS `Lambda` and `EC2` execution environments always populate that same environment variable automatically, with genuinely temporary, session-token-bearing credentials, this logic silently discarded the real temporary credentials in favour of the hardcoded pair on every real deployment, which would have caused every `DynamoDB` and `SQS` call to fail authentication in production despite passing every `LocalStack`-backed test. The fix was to gate the hardcoded override on the presence of an explicit `LocalStack` endpoint override instead, the only condition actually unique to the local-emulator case, expressed with a genuinely different code shape in each of the three files rather than the same patch copy-pasted three times.

Second, dce-api's own backend-stats endpoint reported the total number of items stored in `DynamoDB` via a single `COUNT-select` Scan call. A `COUNT-select` scan only counts the single page it reads, typically capped around 1MB of table data; a table larger than that returns a `LastEvaluatedKey` that must be followed with a further scan, or the reported count silently undercounts the table. This defect was found and fixed before it manifested as a visible symptom in this project's own data volume, since the same pagination gap had already caused a real, deployment-triggered undercount in a separate project in this portfolio built against the same AWS SDK. The fix follows `LastEvaluatedKey` across pages and sums every page's `Count`, with two new tests covering the multi-page-summing and scan-failure cases.

Both defects were fixed and re-verified against the live deployment: the credential fix was confirmed via a genuine `DynamoDB` write and `SQS` receive succeeding end to end after redeployment, and the pagination fix was confirmed by a

real `items_in_table` count returned correctly against the deployed table, rather than trusted from the unit test suite alone.

IV. EVALUATION

A. Automated Test Coverage

Table II summarises the automated test suite as of the final verified deployment. All 114 tests pass against both the LocalStack-backed development environment and, for the parts of the suite that exercise real client-construction logic, against the fixes verified against the real AWS deployment described in Section III-D.

TABLE II. AUTOMATED TEST SUITE BY MODULE

Module	Tests	Focus
sensors	12	dual-rate sampling/dispatch independence
fog	46	ring buffer wraparound, windowed aggregation, alerting
backend/processor	9	DynamoDB item shape, sort-key logic
backend/api	37	internal routing, DynamoDB pagination, thresholds proxy
backend/dashboard	10	reverse-proxy behaviour, invoke-URL resolution

B. Live Load Test

A burst load test sent 300 synthetic messages directly to the real, deployed `dce-hall-agg` queue in 5.47 seconds (approximately 55 messages per second), using synthetic sensor-type names distinct from the five real sensor types so the burst could never land in the DynamoDB partitions the live dashboard reads from. Amazon SQS's own `ApproximateNumberOfMessages` and `ApproximateNumberOfMessagesNotVisible` attributes reported a queue depth of zero immediately after the burst completed, a result that would ordinarily read as a failed send. A direct DynamoDB query against the five synthetic sensor-type partitions, however, confirmed that all 300 messages had genuinely been received, processed, and written by `dce-processor` before the follow-up check even ran.

This is consistent with a documented property of real SQS rather than evidence of a broken pipeline: the queue's own approximate-count attributes are explicitly eventually consistent and can under-report immediately after a burst, a behaviour LocalStack's own, more immediately consistent emulation does not reproduce, and which this evaluation would not have discovered without testing against the real service.

C. Live Deployment Health

Polling the deployed API's own `/api/health` endpoint twice, fifteen seconds apart, returned `freshness_age_seconds` of 7.429 and 3.357 respectively, and `items_in_table` counts consistent with a genuinely live, continuously updating pipeline rather than static or cached data. All AWS resources (the DynamoDB table, the SQS queue, both Lambda functions, the API Gateway API, and the EC2 instance) were independently confirmed healthy via direct AWS CLI calls, not merely trusted from the application's own self-reported status.

Operationally, every backend component other than the EC2 instance runs within AWS's request-based free-tier allowances at this project's data volume; the EC2 instance itself is the only continuously billed resource, and it can be stopped between demonstration sessions without affecting the dashboard or its API, since neither depends on it being

available except for the gateway field of `/api/health` and freshly generated sensor data.

D. Limitations

Three limitations of this evaluation are worth stating explicitly instead of leaving them implicit. First, the fog and sensor tier still runs on a single EC2 instance and therefore remains a single point of failure for that tier specifically, even though the backend behind it is fully redundant AWS-managed infrastructure; the critical analysis in Section II-F explains why converting this tier to a fully serverless model was deferred rather than attempted, but the resulting availability gap is real and unresolved, not merely theoretical. Second, the AWS Academy Learner Lab account used for this deployment is itself session-based rather than persistent, so the sustained, multi-day uptime evaluation a production deployment would warrant was not possible within the account's own constraints; the evaluation in this section reflects verified behaviour within a single continuous session, not long-run reliability. Third, both Lambda functions share one broadly scoped IAM role instead of two least-privilege roles, a constraint of the Learner Lab account and not a deliberate security design, as discussed in Section II-G; this is an actual limitation of what could be independently verified about the deployment's security posture, not evidence that the least-privilege alternative was evaluated and rejected.

V. CONCLUSION

This project set out to build a complete fog and edge computing pipeline for data center environmental monitoring, with a configurable sensor and fog layer, a scalable cloud backend split across two independently deployed Lambda functions, and genuine deployment and testing on a public cloud platform, and to critically analyse the architectural decisions taken at each layer against the alternatives available. All three layers were implemented, tested, and deployed to a real AWS account rather than a local simulation alone.

The most significant finding of this project is not architectural but methodological: local testing against a cloud emulator, however thorough, is not a substitute for testing against the real target platform. Both defects discovered during the real AWS deployment, the credential-handling bug and the DynamoDB pagination bug, passed the full local test suite and a full LocalStack-backed integration test cleanly, and would have shipped undetected had the project stopped at the local-simulation stage. The live load test's own eventual-consistency finding reinforces the same point from a different angle: even a passing local test cannot exercise a platform behaviour, like SQS's approximate-count consistency model, that the local emulator does not itself reproduce.

The most difficult part of this project was not writing application code but discovering the AWS Academy Learner Lab account's own platform and IAM constraints, several of which, the single shared LabRole and the session-based rather than persistent account lifetime, shaped design decisions directly rather than being incidental. Future work would include converting the fog and sensor tier itself to a fully event-driven, scheduled-Lambda model rather than a continuously running EC2 instance, and, if the account restrictions were lifted, provisioning two separate least-privilege IAM roles for `dce-processor` and `dce-api` instead of the single shared LabRole this deployment was constrained to.

REFERENCES

- [1] C.-J. M. Liang, J. Liu, L. Luo, A. Terzis, and F. Zhao, "RACNet: A high-fidelity data center sensing network," in Proc. 7th ACM Conf.

Embedded Networked Sensor Systems (SenSys '09), Berkeley, CA, USA, 2009, pp. 15–28.

- [2] M. G. Rodriguez, L. Ortiz Uriarte, Y. Jia, K. Yoshii, R. Ross, and P. H. Beckman, “Wireless sensor network for data-center environmental monitoring,” in Proc. 2011 Fifth Int. Conf. Sensing Technology (ICST), Palmerston North, New Zealand, 2011, pp. 533–537.
- [3] P. Mell and T. Grance, “The NIST Definition of Cloud Computing,” National Institute of Standards and Technology, Gaithersburg, MD, USA, Special Publication 800-145, 2011.
- [4] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, “Fog computing and its role in the Internet of Things,” in Proc. 1st ACM Mobile Cloud Computing Workshop (MCC '12), Helsinki, Finland, 2012, pp. 13–16.
- [5] J. Giacomoni, T. Moseley, and M. Vachharajani, “FastForward for efficient pipeline parallelism: A cache-optimized concurrent lock-free queue,” in Proc. 13th ACM SIGPLAN Symp. Principles and Practice of Parallel Programming (PPoPP '08), 2008, pp. 43–52.
- [6] L. Lamport, “Concurrent reading and writing,” *Commun. ACM*, vol. 20, no. 11, pp. 806–811, 1977.
- [7] P. T. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec, “The many faces of publish/subscribe,” *ACM Comput. Surv.*, vol. 35, no. 2, pp. 114–131, Jun. 2003.
- [8] A. Carzaniga, D. S. Rosenblum, and A. L. Wolf, “Design and evaluation of a wide-area event notification service,” *ACM Trans. Comput. Syst.*, vol. 19, no. 3, pp. 332–383, Aug. 2001.
- [9] G. DeCandia et al., “Dynamo: Amazon’s highly available key-value store,” in Proc. 21st ACM SIGOPS Symp. Operating Systems Principles (SOSP '07), 2007, pp. 205–220.
- [10] M. Shahradd, J. Balkind, and D. Wentzlaff, “Architectural implications of function-as-a-service computing,” in Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO-52), Columbus, OH, USA, 2019, pp. 1063–1075.
- [11] A. Akbulut and H. G. Perros, “Software versioning with microservices through the API gateway design pattern,” in Proc. 2019 9th Int. Conf. Advanced Computer Information Technologies (ACIT), Ceske Budejovice, Czech Republic, 2019, pp. 289–292.